Pry

There's also a third-party alternative to the official Ruby debugger and the official irb client called Pry.[2] Pry predates the Ruby 3.1 debugger, and contains many overlapping features, but with a slightly different command-line interface.

To install Pry, you need the Pry gem. Pry also has a plugin system that allows extensions, and we'll be adding the pry-byebug gem that gives debugger step control to Pry. You can add them to a Gemfile:

```
gem "pry"
gem "pry-byebug"
```

The main way to use Pry is by adding the method call binding.pry into a code base. If we do that with the same code we used for the debugger earlier, just replacing require debug with require pry and the binding.break call with binding.pry, we get this:

```
From: /Users/noel/projects/pragmatic/ruby5/Book/code/debugging/pry_test.rb:32 :

    27:   end
    28: end
    29:
    30: register = CashRegister.new(0.05)
    31: register.add_item("pen", 3, 499)
 => 32: binding.pry
    33: register.add_item("paper", 2, register.sale_price(799, 25))
    34: p register.total

[1] pry(main)>
```

That certainly looks familiar. We've got the line of code we're blocked at with a few lines of context surrounding it, and then a command prompt below it.

This command prompt is a full REPL, similar to irb, and you can type in arbitrary Ruby expressions within it, including the creation of new methods or classes. You can also type commands (more on that in a bit), but one very important command to start with is exit, which takes you out of the current frame. If you are at the top level, you exit the program. You can also exit the program at any time by typing !!!.

For all Pry output, if the output runs longer then the current screen, it will act like a paging reader (specifically the Unix less program), which is to say it will pause output at the bottom of the screen with a : prompt and await navigation. The space bar will move you one screen forward; the letter b will move you one page back. The enter key or j moves you one line forward, and k moves you one line back. You need to type q to quit and go back to regular input.

Like irb, Pry considers itself to be running inside a particular scope, which is self to anything you type at the command prompt and is the target of other Pry commands. The Pry command structure was based on Unix command shell syntax, so you can change the scope to a different object with cd OBJECT. The object in question must be visible to Pry.

As with Unix, cd .. goes back to the previous scope, cd / takes you back to the original top level scope of the session, and cd - toggles you between the last two scopes.

2. http://pry.github.io